

GRASP metaheuristika za dvostepeni transportni problem sa fiksnim troškovima (TS-FCTP)

Mateja Stojanović
Broj indeksa: mi251107

8. jul 2026.

Sažetak

U ovom radu razmatra se Dvostepeni transportni problem sa fiksnim troškovima (engl. *Two-Stage Fixed-Charge Transportation Problem*, TS-FCTP), koji modeluje distribuciju robe od dobavljača do distributivnih centara i od distributivnih centara do kupaca uz varijabilne i fiksne troškove transporta. Predložena je i implementirana metaheuristika zasnovana na GRASP pristupu (engl. *Greedy Randomized Adaptive Search Procedure*) sa nizom proširenja: reaktivnim parametrom α , lokalnom pretragom tipa VND sa četiri susedstva, elitnim bazenom rešenja, prelaskom putanjom (engl. *path relinking*) i perturbacijom zasnovanom na *ruin-and-recreate* strategiji. Implementacija je dodatno vektorizovana (NumPy) i paralelizovana (**multiprocessing**) na nivou nezavisnih GRASP iteracija, čime je omogućeno izvršavanje algoritma više puta ($k = 3$) po instanci u razumnom vremenu i dobijanje statistički smislenih pokazatelja ($\overline{gap}$, σ), umesto jednog izvršavanja po instanci.

Algoritam je testiran na skupu od 60 instanci malih, srednjih i velikih dimenzija (po 20 u svakoj grupi, $k = 3$ izvršavanja po instanci) i upoređen sa rešenjima dobijenim CPLEX solverom. Prosečno odstupanje od CPLEX referentne vrednosti iznosi $\overline{gap} = 9,24\%$ ($\sigma = 0,48\%$) na malim, $\overline{gap} = 18,72\%$ ($\sigma = 0,71\%$) na srednjim i $\overline{gap} = 29,19\%$ ($\sigma = 0,92\%$) na velikim instancama.

Ključne reči: dvostepeni transportni problem sa fiksnim troškovima, GRASP, metaheuristika, lokalna pretraga, path relinking, vektorizacija, paralelizacija, kombinatorna optimizacija.

Sadržaj

1	Opis problema i matematička formulacija	3
1.1	Definicija problema	3
1.2	Skupovi i parametri	3
1.3	Promenljive	3
1.4	Funkcija cilja	3
1.5	Ograničenja	4
1.6	Značenje funkcije cilja i uslova	4
1.7	Oblasti primene	4
1.8	Postojeći načini rešavanja	5
2	Metaheuristika GRASP i njena adaptacija	5
2.1	Osnove GRASP-a	5
2.2	Implementacija	5
2.2.1	Reprezentacija rešenja – klasa <code>Solution</code>	5
2.2.2	Procedura REPAIR	6
2.2.3	Faza konstrukcije – klasa <code>Construction</code>	7
2.2.4	Lokalna pretraga – klasa <code>LocalSearch</code> (VND)	7
2.2.5	Elitni bazen – klasa <code>ElitePool</code>	8
2.2.6	Prelazak putanjom – klasa <code>PathRelinking</code>	8
2.2.7	Perturbacija – <i>ruin-and-recreate</i> (shake)	9
2.2.8	Reaktivni parametar α	9
2.2.9	Vektorizacija	9
2.2.10	Paralelizacija – batch nezavisnih iteracija	9
2.2.11	Praćenje t_i i $eval_i$ za MA izveštaj	10
2.2.12	Kompletan pseudokod GRASP algoritma	12
3	Rezultati testiranja i analiza	13
3.1	Test instance	13
3.2	Referentna rešenja (CPLEX)	13
3.3	Podešavanja GRASP-a, platforma i metodologija izveštaja	13
3.4	Male instance	13
3.5	Srednje instance	14
3.6	Velike instance	15
3.7	Analiza rezultata	15
3.7.1	Male instance	15
3.7.2	Srednje instance	16
3.7.3	Velike instance	16
3.7.4	Zajednički zaključci	16
4	Zaključak	17

1 Opis problema i matematička formulacija

1.1 Definicija problema

Dvostepeni transportni problem sa fiksnim troškovima (TS-FCTP) je problem kombinatorne optimizacije koji se javlja u planiranju logističkih mreža. Mreža se sastoji od tri sloja čvorova, povezanih u dve etape transporta:

$$i \rightarrow j \rightarrow k,$$

gde su:

- $i \in I$ – **dobavljači** (eng. *suppliers*), sa raspoloživom ponudom s_i ;
- $j \in J$ – **distributivni centri** (DC);
- $k \in K$ – **kupci** (eng. *customers*), sa potražnjom d_k .

Za svaku vezu u mreži postoje varijabilni trošak (proporcionalan protoku) i fiksni trošak koji se plaća jednom ukoliko se veza koristi, tj. ukoliko je protok na njoj pozitivan. Cilj je minimizacija ukupnog troška transporta uz poštovanje ograničenja kapaciteta dobavljača i ispunjenje potražnje kupaca.

1.2 Skupovi i parametri

- I – skup dobavljača (izvora), $|I| = I$;
- J – skup distributivnih centara, $|J| = J$;
- K – skup kupaca, $|K| = K$;
- s_i – raspoloživa ponuda dobavljača i ;
- d_k – potražnja kupca k ;
- b_{ij} – varijabilni trošak transporta od dobavljača i do centra j ;
- f_{ij} – fiksni trošak aktiviranja veze (i, j) ;
- c_{jk} – varijabilni trošak transporta od centra j do kupca k ;
- g_{jk} – fiksni trošak aktiviranja veze (j, k) .

1.3 Promenljive

- $x_{ij} \geq 0$ – količina koja se transportuje od dobavljača i do centra j ;
- $y_{jk} \geq 0$ – količina koja se transportuje od centra j do kupca k ;
- $z_{ij} \in \{0, 1\}$ – indikator da li se koristi veza (i, j) ;
- $w_{jk} \in \{0, 1\}$ – indikator da li se koristi veza (j, k) .

1.4 Funkcija cilja

Minimizuje se suma varijabilnih i fiksnih troškova u obe etape:

$$\min \sum_{i \in I} \sum_{j \in J} (b_{ij}x_{ij} + f_{ij}z_{ij}) + \sum_{j \in J} \sum_{k \in K} (c_{jk}y_{jk} + g_{jk}w_{jk}).$$

1.5 Ograničenja

Ograničenje ponude dobavljača

$$\sum_{j \in J} x_{ij} \leq s_i, \quad \forall i \in I.$$

Zadovoljenje potražnje kupaca

$$\sum_{j \in J} y_{jk} = d_k, \quad \forall k \in K.$$

Balans protoka u distributivnim centrima Ukupni ulaz u centar mora biti jednak ukupnom izlazu – DC ne skladišti niti stvara robu, već je samo prosleđuje:

$$\sum_{i \in I} x_{ij} - \sum_{k \in K} y_{jk} = 0, \quad \forall j \in J.$$

Aktivacija fiksnih troškova (Big-M povezivanje) Ako se veza koristi (protok je pozitivan), odgovarajuća binarna promenljiva mora biti 1:

$$\begin{aligned} x_{ij} &\leq s_i z_{ij}, \quad \forall i \in I, \forall j \in J, \\ y_{jk} &\leq d_k w_{jk}, \quad \forall j \in J, \forall k \in K. \end{aligned}$$

1.6 Značenje funkcije cilja i uslova

- $b_{ij}x_{ij}$ i $c_{jk}y_{jk}$ su varijabilni troškovi proporcionalni protoku, dok su $f_{ij}z_{ij}$ i $g_{jk}w_{jk}$ fiksni troškovi koji se plaćaju jednom, ako se veza uopšte koristi.
- Ograničenja ponude obezbeđuju da nijedan dobavljač ne šalje više robe nego što ima na raspolaganju.
- Ograničenja potražnje garantuju da se zahtev svakog kupca tačno zadovolji.
- Balans u centrima znači da DC ne može praviti zalihe niti stvarati robu – samo prosleđuje protok dalje.
- Big-M ograničenja povezuju kontinualne tokove sa binarnim indikatorima, čime se modeluje fiksni trošak aktivacije veze.

1.7 Oblasti primene

TS-FCTP se javlja u raznim oblastima:

- **Upravljanje lancima snabdevanja** – planiranje toka robe od fabrika do magacina i do maloprodajnih objekata;
- **Distribucija električne energije** – alokacija energije od elektrana do transformatorskih stanica i do potrošača;
- **Telekomunikacije** – rutiranje podataka kroz čvorove mreže;
- **Humanitarna logistika** – distribucija pomoći od centralnih magacina do regionalnih centara i do krajnjih korisnika.

1.8 Postojeći načini rešavanja

Problem TS-FCTP pripada klasi NP-teških problema. Postojeći pristupi uključuju:

- **Tačni metodi:** MIP formulacija rešena komercijalnim solverima (npr. IBM ILOG CPLEX, Gurobi) uz primenu *branch-and-bound* i *branch-and-cut* algoritama. Za male instance postiže se optimalnost, ali za srednje i velike instance vreme rešavanja eksponencijalno raste.
- **Lagranževa relaksacija:** Relaksuju se ograničenja balansa ili kapaciteta uz Lagranževe multiplikatore koji se ažuriraju *subgradijentnom metodom*. Dobijaju se dobre donje granice.
- **Tabu pretraga:** Iterativna lokalna pretraga sa listom zabrana za sprečavanje cikliranja. Efektivna za transportne probleme.
- **Genetski algoritmi / evolutivne metaheuristike:** Kombinuju više rešenja putem ukrštanja i mutacije.
- **GRASP:** Metaheuristika opisana u ovom radu, zasnovana na pohlepnoj nasumičnoj konstrukciji i lokalnoj pretrazi.

2 Metaheuristika GRASP i njena adaptacija

2.1 Osnove GRASP-a

GRASP (*Greedy Randomized Adaptive Search Procedure*) je iterativna metaheuristika koju su uveli Feo i Resende (1995). Svaka iteracija se sastoji od dve faze:

1. **Faza konstrukcije:** gradi se dopustivo rešenje pohlepno-nasumičnim pristupom. U svakom koraku formira se *Restricted Candidate List* (RCL) – skup kandidata čija je ocena u opsegu $[\min, \min + \alpha(\max - \min)]$ – i nasumično se bira jedan element RCL-a. Parametar $\alpha \in [0, 1]$ kontroliše stepen nasumičnosti: $\alpha = 0$ odgovara čistoj pohlepnoj heuristici, a $\alpha = 1$ nasumičnom izboru.
2. **Faza lokalne pretrage:** dobijeno rešenje se poboljšava lokalnom pretragom u skupu susednih rešenja.

Algoritam se ponavlja unapred zadati broj iteracija i pamti globalno najbolje pronađeno rešenje.

2.2 Implementacija

Trenutna implementacija (folder `GRASP_part_parallel/` u repozitorijumu projekta) sastoji se iz tri fajla: `utils.py` (učitavanje instance, infrastruktura za multi-run eksperimente i izveštaj), `GRASP.py` (ceo algoritam) i `main.py` (CLI ulaz). U nastavku je detaljno opisana svaka klasa/funkcija iz `GRASP.py`, potom kompletan pseudokod algoritma.

2.2.1 Reprezentacija rešenja – klasa `Solution`

Rešenje se predstavlja parom matrica toka i skupom otvorenih DC-ova:

- $x \in \mathbb{Z}_{\geq 0}^{I \times J}$ – tokovi I. etape (dobavljač $\rightarrow$ DC);
- $y \in \mathbb{Z}_{\geq 0}^{J \times K}$ – tokovi II. etape (DC $\rightarrow$ kupac);

- $\text{open_dc} \subseteq \{1, \dots, J\}$ – skup otvorenih DC-ova (DC j je otvoren ako $\sum_i x_{ij} > 0$ ili $\sum_k y_{jk} > 0$).

Klasa ima tri metode:

- `copy()` – duboka kopija rešenja (kopiraju se x , y , cena i `open_dc`); koristi se jer se lokalna pretraga grana kroz probne (*trial*) kopije koje se odbacuju ako ne donesu poboljšanje.
- `compute_cost()` – računa ukupnu cenu rešenja ($\sum b_{ij}x_{ij} + \sum c_{jk}y_{jk} + \sum f_{ij}[x_{ij} > 0] + \sum g_{jk}[y_{jk} > 0]$) i skup otvorenih DC-ova, u potpunosti vektorizovano preko NumPy-ja (`x.sum(axis=0) > 0 | y.sum(axis=1) > 0`, umesto petlje po j sa dva poziva sume po DC-u). Svaki poziv ove metode uvećava globalni brojač *eval* (broj poziva funkcije cilja, videti odeljak o MA izveštaju).
- `is_valid()` – proverava da li je zadovoljena potražnja svakog kupca, da li nijedan dobavljač nije prekoračio kapacitet i da li važi balans toka u svakom DC-u.

Ključna arhitekturna odluka: tokom pretrage se direktno modifikuju isključivo y -tokovi (II. etapa); x -tokovi (I. etapa) se pri svakoj promeni y -toka iznova rekonstruišu pohlepnom procedurom REPAIR. Ovaj pristup smanjuje dimenzionalnost prostora pretrage jer algoritam nikad eksplicitno ne upravlja x -promenljivama.

2.2.2 Procedura REPAIR

Za svaki DC j sa pozitivnim ukupnim prilivom od kupaca ($\text{demand}_j = \sum_k y_{jk}$), dobavljači se rangiraju po oceni koja kombinuje varijabilni i amortizovani fiksni trošak, $\text{score}_i = b_{ij} + f_{ij}/\text{demand}_j$, a zatim se tokovi pohlepno dodeljuju u tom redosledu dok se zahtev DC-a ne zadovolji. Ako neki DC ne može biti u potpunosti snabdeven (nedostatak ukupnog kapaciteta), procedura vraća neuspeh.

Algorithm 1: Repair(sol, D)

```

1  $sol.x \leftarrow 0$ ;
2  $rem\_s \leftarrow D.s.copy()$ ;
3 for  $j = 1$  to  $J$  do
4    $\text{demand}_j \leftarrow \sum_k y_{jk}$ ;
5   if  $\text{demand}_j = 0$  then continue;
6   Sortiraj dobavljače po  $\text{score}_i = b_{ij} + f_{ij}/\text{demand}_j$ ;
7    $rem \leftarrow \text{demand}_j$ ;
8   for svakog  $i$  u sortiranom redosledu do
9      $q \leftarrow \min(\text{rem\_s}[i], rem)$ ;
10     $x_{ij} += q$ ;  $rem\_s[i] -= q$ ;  $rem -= q$ ;
11    if  $rem = 0$  then break;
12  end
13  if  $rem > 0$  then return neuspešno;
14 end
15 return uspešno

```

Napomena o performansama: REPAIR je jedina preostala nevektorizovana (sekvencijalna) tačka u lokalnoj pretrazi – svaki DC deli tačno izbalansiran, ograničen kapacitet dobavljača (ukupna ponuda je uvek jednaka ukupnoj potražnji), pa bi inkrementalno prepravljanje samo pogodnih DC-ova moglo promeniti da li REPAIR uspeva za pojedine kandidate –

taj rizik po korektnost nije preuzet.

2.2.3 Faza konstrukcije – klasa Construction

Kupci se obrađuju u opadajućem redosledu po potražnji. Za svakog kupca k , u svakom koraku se formira RCL od parova (i, j) sa ocenom:

$$\text{score}(i, j) = b_{ij} + c_{jk} + \frac{f_{ij}}{\text{need}} \cdot [x_{ij} = 0] + \frac{g_{jk}}{\text{need}} \cdot [y_{jk} = 0],$$

gde je need preostala potražnja kupca k u tekućem koraku. RCL sadrži sve parove čija je ocena $\leq \text{score}_{\min} + \alpha(\text{score}_{\max} - \text{score}_{\min})$, a iz njega se nasumično bira par kome se dodeljuje tok $q = \min(\text{rem}_s, \text{need})$.

Cela $I \times J$ matrica ocena za tekućeg kupca gradi se i maskira vektorizovano preko NumPy broadcast-a (umesto dvostruke petlje `for i: for j:` koja generiše listu tuple-ova i sortira je), što je za velike instance (I, J reda stotina) dominantan doprinos ubrzanju konstrukcije – u merenjima na srednjoj instanci construction je posle vektorizacije $> 100\times$ brži po pozivu.

Algorithm 2: GreedyRandomizedConstruction(D, α)

```

1  $sol \leftarrow$  prazno rešenje;
2 for  $k$  u opadajućem redosledu po  $d_k$  do
3    $need \leftarrow d_k$ ;
4   while  $need > 0$  do
5     Vektorizovano izračunaj  $I \times J$  matricu ocena  $\text{score}(i, j)$ ;
6     Maskiraj dobavljače sa  $\text{rem}_s[i] \leq 0$  (ocena =  $\infty$ );
7      $\text{score}_{\min} \leftarrow \min$ ,  $\text{score}_{\max} \leftarrow \max$  (nad konačnim vrednostima);
8      $\text{RCL} \leftarrow \{(i, j) : \text{score}(i, j) \leq \text{score}_{\min} + \alpha(\text{score}_{\max} - \text{score}_{\min})\}$ ;
9     Nasumično izaberi  $(i^*, j^*)$  iz RCL;
10     $q \leftarrow \min(\text{rem}_s[i^*], need)$ ;
11     $x_{i^*j^*} += q$ ;  $y_{j^*k} += q$ ;  $\text{rem}_s[i^*] -= q$ ;  $need -= q$ ;
12  end
13 end
14 Izračunaj cenu  $sol$ ;
15 return  $sol$ 
```

2.2.4 Lokalna pretraga – klasa LocalSearch (VND)

Primenjuje se *Variable Neighborhood Descent* (VND) sa četiri susedstva, metodom *first improvement* – prvi pronađeni pomak koji smanjuje cenu se prihvata i pretraga se vraća na prvo susedstvo. Sve četiri metode koriste vektorizovanu (NumPy) evaluaciju kandidata: umesto petlje `for k in range(K)` koja za svakog kupca traži ko ga trenutno opslužuje (`next(j: y[j,k]>0)`), traži se jednim pozivom `np.argmax(y > 0, axis=0)` za sve kupce odjednom.

1. **Preraspoređivanje kupca** (`_reassign_customer`): za svakog kupca k koji je dodeljen DC-u j_{from} , nasumično se isprobava do 10 alternativnih DC-ova j_{to} (`random.sample`, umesto punog `shuffle` liste svih J DC-ova). Ako bi premeštanje smanjilo ukupnu cenu (jeftinija procena $c_{jk}d_k + g_{jk}$), premešta se ceo tok kupca (uz REPAIR i proveru stvarne cene).

2. **Dodavanje DC-a** (`_add_dc`): isprobava se do 15 zatvorenih DC-ova kao kandidati za otvaranje. Za svakog kandidata j_{new} , vektorizovano (numpy fensi-indeksiranje) se određuje skup kupaca kojima bi bilo jeftinije preko j_{new} , i svi se odjednom presmeravaju.
3. **Uklanjanje DC-a** (`_remove_dc`): isprobava se do 5 otvorenih DC-ova za zatvaranje; za svakog pogođenog kupca vektorizovano se bira najjeftinija preostala alternativa ($\arg \min$ po koloni matrice troška oblika $|\text{alts}| \times |\text{pogođeni kupci}|$).
4. **Zamena DC-a** (`_swap_dc`): isprobava se do 30 nasumičnih parova (otvoren DC, zatvoren DC) i izvršava zamena (svi kupci otvorenog DC-a presmeravaju se na zatvoreni) ako je poboljšanje moguće.

Posle svakog uspešnog pomaka susedstvo se resetuje na prvo (klasičan VND ciklus); ako nijedno od četiri susedstva ne poboljša rešenje `max_no_improve` = 6 puta uzastopno, pretraga se zaustavlja.

Algorithm 3: VND($D, sol, \text{max_no_improve} = 6$)

```

1  Izračunaj cenu sol;
2  no_improve  $\leftarrow$  0;
3  while no_improve < max_no_improve do
4    if ReassignCustomer(sol) then
5      | no_improve  $\leftarrow$  0; continue
6    else if AddDC(sol) then
7      | no_improve  $\leftarrow$  0; continue
8    else if RemoveDC(sol) then
9      | no_improve  $\leftarrow$  0; continue
10   else if SwapDC(sol) then
11     | no_improve  $\leftarrow$  0; continue
12   else
13     | no_improve += 1
14   end
15 end
16 return sol

```

2.2.5 Elitni bazen – klasa ElitePool

Čuva se skup od do 12 najkvalitetnijih i međusobno raznolikih rešenja. Rešenje se ne dodaje u bazen ako u njemu već postoji rešenje sa istim skupom otvorenih DC-ova (`open_dc`) i sličnom cenom (razlika < 50) – na taj način bazen ne postaje prepun skoro identičnih rešenja. Metoda `best()` vraća rešenje najniže cene, a `random()` nasumično rešenje iz bazena; oba se koriste kao ulaz za *path relinking*.

2.2.6 Prelazak putanjom – klasa PathRelinking

Svakih 8 iteracija, uzimaju se dva rešenja iz elitnog bazena (najbolje i nasumično) i evaluiraju se **oba** kandidata za novi skup otvorenih DC-ova: **unija** i **presek** njihovih skupova otvorenih DC-ova. Za svaki od ta dva kandidatska skupa, svaki kupac se dodeljuje DC-u iz tog skupa sa najnižom procenjenom cenom ($c_{jk}d_k + g_{jk}$), zatim se primenjuje REPAIR i potom VND; zadržava se bolji od dva rezultata.

Algorithm 4: PathRelinking(a, b, D)

```
1  $best \leftarrow \text{null}; \quad best\_cost \leftarrow +\infty;$ 
2 for  $dc\_set \in \{a.open\_dc \cup b.open\_dc, a.open\_dc \cap b.open\_dc\}$  do
3   if  $dc\_set = \emptyset$  then continue;
4    $trial \leftarrow$  prazno rešenje;
5   for  $k = 1$  to  $K$  do
6      $j^* \leftarrow \arg \min_{j \in dc\_set} (c_{jk}d_k + g_{jk});$ 
7      $y_{j^*k} \leftarrow d_k;$ 
8   end
9   if  $REPAIR(trial, D)$  then
10     $trial \leftarrow VND(D, trial);$ 
11    if  $trial.cost < best\_cost$  then  $best \leftarrow trial; best\_cost \leftarrow trial.cost;$ 
12  end
13 end
14 return  $best$  (ili  $a$ , ako oba propadnu)
```

2.2.7 Perturbacija – *ruin-and-recreate* (shake)

Ako se 20 uzastopnih iteracija ne pronađe poboljšanje najboljeg rešenja, primenjuje se perturbacija: DC-ovi se rangiraju po prosečnom varijabilnom trošku po jedinici protoka, uklanja se 50% najskupljih, a kupci koji su im bili dodeljeni presmeravaju se na preostale (otvorene) DC-ove po najnižoj proceni troška. Posle REPAIR-a i provere dopustivosti primenjuje se VND; ako je novo rešenje bolje i dopustivo, prihvata se kao novo najbolje. Brojač zastoja se resetuje posle svakog pokušaja shake-a (bez obzira na uspeh), čime se sprečavaju uzastopni shake-ovi iz istog stanja.

2.2.8 Reaktivni parametar α

Ako je prošlo više od 15 iteracija bez poboljšanja najboljeg rešenja, α se uzorkuje iz $[0,4, 0,7]$ radi diversifikacije pretrage (veći RCL, više nasumičnosti). Inače, $\alpha \in [0,05, 0,5]$ radi intenzifikacije (manji RCL, bliže pohlepnom izboru).

2.2.9 Vektorizacija

Profajliranjem (cProfile) je utvrđeno da su, pre optimizacije, dva mesta dominirala vremenom izvršavanja: (1) konstrukcija – $I \times J$ kandidata generisanih Python petljom po jedinici potražnje, i (2) `compute_cost()` – računanje skupa otvorenih DC-ova Python petljom od J elemenata sa po dva poziva `np.sum()`. Oba mesta su zamenjena NumPy vektorizovanim ekvivalentima (broadcast, boolean maske), bez promene rezultata pretrage – ista RCL/α logika i ista *first-improvement* pravila, samo izračunata bržim putem. Na srednjoj test instanci ovo je local search ubrzalo $\approx 1,75\times$ (sa 12,8 s na 7,3 s za 10 poziva construction+local-search).

2.2.10 Paralelizacija – batch nezavisnih iteracija

GRASP petlja je inherentno sekvencijalna (elitni bazen, praćenje najboljeg rešenja, *path relinking* i shake zavise od deljenog stanja), ali faza konstrukcije i lokalne pretrage jedne iteracije zavisi samo od parametra α i nepromenjenih podataka instance – potpuno je

nezavisna od ostalih iteracija. Zato je moguće paralelizovati batch od n_{workers} iteracija odjednom preko `multiprocessing.Pool`:

- Podaci instance se svakom *worker* procesu šalju tačno jednom, preko `initializer` funkcije `_init_worker` (ne pri svakom pozivu zadatka).
- Funkcija `_construct_and_improve(alpha)` u worker procesu radi construction + local search i vraća samo rezultat (x, y , cena, `open_dc`, broj poziva funkcije cilja) – ne šalje nazad cost-matrice instance, koje su nepromenjene i već dostupne u worker procesu.
- Posle svakog batch-a, elitni bazen, praćenje najboljeg rešenja, *path relinking* i shake primenjuju se **sekvencijalno**, istim redosledom kao u serijskoj verziji – jedina razlika je da su rešenja u batch-u već gotova (izračunata paralelno) pre te sekvencijalne obrade.
- Ako je $n_{\text{workers}} \leq 1$, koristi se u potpunosti originalni serijski kod (`_run_serial`) – ponašanje je tada identično algoritmu bez paralelizacije.

Algorithm 5: GRASP_Parallel(D, T, n_{workers}) – razlika u odnosu na serijsku verziju

```

1 Pokreni Pool od  $n_{\text{workers}}$  procesa, pošalji im  $D$  jednom;
2  $it \leftarrow 0$ ;
3 while  $it < T$  do
4     batch  $\leftarrow \min(n_{\text{workers}}, T - it)$ ;
5     Generiši batch vrednosti  $\alpha$  (reaktivno, kao u serijskoj verziji);
6     results  $\leftarrow$  paralelno izračunaj GreedyRandomizedConstruction+VND za
        svaki  $\alpha$  iz batch-a;
7     for svaki sol iz results (sekvencijalno, istim redosledom kao serijska GRASP
        petlja) do
8         Ažuriraj elitni bazen, najbolje rešenje, (svakih 8) path relinking, (posle 20
            zastoja) shake;
9          $it += 1$ ;
10    end
11 end
12 Zatvori Pool;
13 return najbolje pronađeno rešenje

```

Broj *worker* procesa se podrazumevano bira kao `cpu_count() - 1` (ostavlja se jedno jezgro slobodno), a može se eksplicitno podesiti preko CLI opcije `-workers`.

2.2.11 Praćenje t_i i $eval_i$ za MA izveštaj

Da bi izveštaj (odeljak 3) bio potpun po specifikaciji, implementacija prati, za svako izvršavanje:

- t_i – `GRASP.run()` interno beleži `time.time()` u tačnom trenutku kada se najbolje rešenje poboljša (u glavnoj petlji, u *path relinking*-u i u shake-u), i vraća tu vrednost kao deo rezultata. (Ranija verzija koda je ovde imala grešku – t_i je bio postavljen na ukupno vreme izvršavanja, bez stvarnog praćenja trenutka poboljšanja; to je ispravljeno.)

- $eval_i$ – globalni brojač uvećan pri svakom pozivu `Solution.compute_cost()`. U serijskom režimu čita se direktno na kraju izvršavanja; u paralelnom režimu svaki *worker* proces ima sopstveni (nezavisan) brojač koji se resetuje na početku svakog zadatka i vraća kao deo rezultata, a glavni proces ga sabira sa sopstvenim pozivima (*path relinking*, *shake*).
- $iter_i$ – broj iteracija je fiksiran unapred (nema kriterijuma ranog zaustavljanja), pa je jednak zadatom broju iteracija T za datu grupu instanci (videti odeljak 3.3).
- $cache_i$ – u implementaciji ne postoji mehanizam kešfiguriranja (*memoizacije*) poziva funkcije cilja, pa ovo (opciono) polje nije primenjivo i izostavljeno je iz izveštaja.

2.2.12 Kompletan pseudokod GRASP algoritma

Algorithm 6: GRASP(D, T)

Input: Podaci instance D , broj iteracija T

Output: Najbolje pronađeno rešenje sol^* , vreme t_i nalaska sol^* , broj evaluacija $eval$

```

1  $sol^* \leftarrow \emptyset$ ;  $best\_cost \leftarrow +\infty$ ;  $last\_imp \leftarrow 0$ ;  $t_i \leftarrow 0$ ;  $E \leftarrow \emptyset$  (elitni bazen,
   veličina 12);
2 for  $it = 0$  to  $T - 1$  do
3   if  $it - last\_imp > 15$  then
4      $\alpha \leftarrow \mathcal{U}(0,4, 0,7)$ ;
5   else
6      $\alpha \leftarrow \mathcal{U}(0,05, 0,5)$ ;
7   end
8    $sol \leftarrow \text{GreedyRandomizedConstruction}(D, \alpha)$ ;
9    $sol \leftarrow \text{VND}(D, sol)$ ;
10   $E.add(sol)$ ;
11  if  $sol.cost < best\_cost$  then
12     $sol^* \leftarrow sol$ ;  $best\_cost \leftarrow sol.cost$ ;  $last\_imp \leftarrow it$ ;  $t_i \leftarrow \text{time}() - \text{start}$ ;
13  end
14  if  $it \bmod 8 = 0$  and  $|E| \geq 2$  then
15     $cand \leftarrow \text{PathRelinking}(E.best(), E.random(), D)$ ;
16    if  $cand.cost < best\_cost$  then
17       $sol^* \leftarrow cand$ ;  $best\_cost \leftarrow cand.cost$ ;  $last\_imp \leftarrow it$ ;
18       $t_i \leftarrow \text{time}() - \text{start}$ ;
19    end
20  end
21  if  $it - last\_imp > 20$  then
22     $sol' \leftarrow \text{RuinAndRecreate}(sol^*, D)$ ;
23    if  $REPAIR(sol', D)$  and  $sol'.is\_valid()$  then
24       $sol' \leftarrow \text{VND}(D, sol')$ ;
25      if  $sol'.is\_valid()$  and  $sol'.cost < best\_cost$  then
26         $sol^* \leftarrow sol'$ ;  $best\_cost \leftarrow sol'.cost$ ;  $t_i \leftarrow \text{time}() - \text{start}$ ;
27      end
28    end
29     $last\_imp \leftarrow it$ ; // sprečava uzastopne shake-ove
30  end
31 return  $sol^*, t_i, eval$ 

```

Paralelna verzija (Algoritam 5) izvršava telo petlje po it na identičan način, s tim što se GreedyRandomizedConstruction+VND za grupu od n_{workers} uzastopnih vrednosti it računaju paralelno pre nego što se sekvencijalni deo (ažuriranje E , sol^* , path relinking, shake) primeni na svaki rezultat iz batch-a, tim redosledom.

3 Rezultati testiranja i analiza

3.1 Test instance

Za testiranje je korišćen skup od 60 instanci podeljenih u tri grupe (generisane koordinatnim modelom – čvorovi nasumično u kvadratu $[-400, 400]^2$, varijabilni troškovi su Euklidske distance, fiksni troškovi su višekratnici tih distanci, $r_f \in [10, 15]$ za I. etapu i $r_g \in [5, 10]$ za II. etapu):

- **Male** (small, 20 instanci): $I \in [4, 18]$, $J \in [8, 36]$, $K \in [16, 80]$.
- **Srednje** (medium, 20 instanci): $I \in [20, 60]$, $J \in [40, 120]$, $K \in [80, 275]$.
- **Velike** (large, 20 instanci): $I \in [70, 160]$, $J \in [140, 320]$, $K \in [280, 800]$.

Svaki dobavljač ima kapacitet $s_i = 80$; potražnja kupaca $d_k \in [10, 30]$ je generisana nasumično, a zatim normalizovana tako da ukupna potražnja tačno odgovara ukupnoj ponudi ($\sum_k d_k = \sum_i s_i$).

3.2 Referentna rešenja (CPLEX)

Referentna rešenja dobijena su CPLEX solverom (IBM ILOG CPLEX) korišćenjem MIP formulacije iz odeljka 1, sa vremenskim limitom od 7200 s i jednom niti. Status **OPTI** označava da je CPLEX dokazao optimalnost; status **FEAS** da je dostignut vremenski limit i vraćeno najbolje pronađeno dopustivo rešenje (bez dokaza optimalnosti) – to je i dalje referentna vrednost korišćena za *Bestsol*, u skladu sa napomenom iz specifikacije izveštaja ("optimalno ili najbolje poznato rešenje").

3.3 Podešavanja GRASP-a, platforma i metodologija izveštaja

Platforma za GRASP: x86_64, 16 CPU jezgara, Python 3.12.3, NumPy 2.4.4. Algoritam je izvršavan paralelno sa $n_{\text{workers}} = \text{cpu_count}() - 1 = 15$ worker procesa.

Svaka instanca je izvršena $k = 3$ puta (seed $42 + 100r$ za r -to izvršavanje, $r = 0, 1, 2$), sa fiksnim brojem iteracija po grupi:

- Male instance: $T = 500$;
- Srednje instance: $T = 200$;
- Velike instance: $T = 100$.

Za svako izvršavanje i pamte se sol_i (najbolja cena), t_i (vreme do prvog nalaska sol_i), $ttot_i$ (ukupno vreme izvršavanja) i $eval_i$ (broj poziva funkcije cilja) – videti odeljak 2.2.7. *Bestsol* se određuje kao CPLEX referentna vrednost ako je poznata i bolja od najboljeg MA rezultata, u suprotnom kao najbolji MA rezultat kroz k izvršavanja; zatim se za svako i računa

$$gap_i = 100 \cdot \frac{|sol_i - Bestsol|}{|Bestsol|}.$$

U tabelama su prikazane srednje vrednosti $\bar{t}$, $\overline{ttot}$, $\overline{eval}$, $\overline{gap}$ i standardna devijacija σ (u %), računata preko $k = 3$ izvršavanja po instanci.

3.4 Male instance

Tabela 1: Rezultati za male instance ($T = 500$, $k = 3$)

#	$I/J/K$	Ref. (CPLEX)	Best MA	$\bar{t}$ (s)	$\overline{ttot}$ (s)	$\overline{eval} (\times 10^3)$	$\overline{gap}$ (%)	σ (%)
1	4/8/16	150 568 (OPTI)	151 577	0.9	3.9	135.6	0.67	0.00
2	4/8/20	156 022 (OPTI)	164 939	1.8	4.9	202.0	6.14	0.30
3	4/10/20	161 224 (OPTI)	165 831	4.0	8.8	305.5	2.87	0.01
4	6/12/24	186 537 (OPTI)	195 722	5.8	9.0	287.9	4.92	0.00
5	6/12/30	213 785 (OPTI)	234 361	2.1	14.0	493.0	10.56	0.82
6	6/15/30	188 954 (OPTI)	206 385	8.4	14.8	485.4	9.65	0.36
7	8/16/32	231 984 (OPTI)	248 473	10.0	13.3	401.7	7.17	0.05
8	8/16/40	202 467 (OPTI)	219 349	8.1	18.7	553.2	9.41	0.89
9	8/20/40	210 740 (OPTI)	224 763	20.0	22.7	612.3	7.09	0.36
10	10/20/40	258 236 (OPTI)	275 995	19.0	28.7	650.4	8.23	0.96
11	10/20/50	286 991 (OPTI)	310 636	17.7	31.2	735.3	8.67	0.38
12	10/25/50	284 643 (FEAS)	314 751	15.4	45.8	1179.3	11.08	0.62
13	12/24/48	278 044 (OPTI)	316 830	31.0	51.1	1011.4	14.26	0.26
14	12/24/60	284 648 (OPTI)	328 715	14.3	40.5	869.3	16.85	1.01
15	12/30/60	315 417 (OPTI)	340 537	34.3	65.6	1273.3	9.03	1.23
16	14/28/56	349 820 (FEAS)	385 816	37.8	65.7	1229.8	10.96	0.55
17	14/28/70	368 942 (FEAS)	408 612	65.5	82.1	1507.8	11.09	0.35
18	16/32/64	349 036 (FEAS)	384 333	39.4	72.6	1227.1	10.50	0.48
19	16/32/80	351 345 (FEAS)	397 712	40.2	88.3	1444.9	13.35	0.17
20	18/36/72	400 600 (FEAS)	446 026	25.4	93.9	1380.8	12.36	0.72
Prosek				20.1	38.8	799.3	9.24	0.48

3.5 Srednje instance

Tabela 2: Rezultati za srednje instance ($T = 200$, $k = 3$)

#	$I/J/K$	Ref. (CPLEX)	Best MA	$\bar{t}$ (s)	$\overline{ttot}$ (s)	$\overline{eval} (\times 10^3)$	$\overline{gap}$ (%)	σ (%)
1	20/40/80	383 558 (FEAS)	421 275	26.8	41.5	558.1	10.95	1.01
2	20/40/100	415 983 (FEAS)	470 055	36.9	69.7	828.1	13.52	0.69
3	20/50/100	400 875 (FEAS)	464 645	48.9	78.3	839.9	17.32	1.03
4	25/50/100	395 660 (FEAS)	464 691	23.3	70.5	706.8	17.75	0.26
5	25/50/125	474 978 (FEAS)	556 570	63.4	106.8	1069.1	18.39	0.88
6	25/60/120	447 777 (FEAS)	528 696	107.2	160.4	1496.8	18.63	0.40
7	30/60/120	474 863 (FEAS)	533 621	58.3	96.5	820.8	13.84	1.10
8	30/60/150	514 195 (FEAS)	619 351	106.4	211.9	1663.8	21.09	0.77
9	30/75/150	503 494 (FEAS)	580 773	134.2	208.5	1413.1	16.21	0.62
10	35/70/140	525 261 (FEAS)	619 496	113.5	185.0	1416.3	18.24	0.22
11	35/70/175	551 427 (FEAS)	647 867	175.9	237.8	1622.1	18.53	0.81
12	40/80/160	478 570 (FEAS)	544 689	172.1	225.8	1352.2	14.94	0.82
13	40/80/200	616 913 (FEAS)	764 595	177.1	439.3	2832.9	24.16	0.15
14	45/90/180	689 812 (FEAS)	829 035	258.9	345.2	2039.4	21.20	0.84
15	45/90/225	557 928 (FEAS)	657 472	193.6	369.1	1956.9	20.05	1.73

Nastavlja se na sledećoj strani

Tabela 2 – nastavak

#	$I/J/K$	Ref. (CPLEX)	Best MA	$\bar{t}$ (s)	$\overline{ttot}$ (s)	$\overline{eval} (\times 10^3)$	$\overline{gap}$ (%)	σ (%)
16	50/100/200	592 227 (FEAS)	721 497	273.0	482.3	2396.1	22.40	0.41
17	50/100/250	612 725 (FEAS)	733 820	292.9	708.0	3534.2	21.36	1.22
18	55/110/220	685 116 (FEAS)	858 810	439.4	590.1	2723.6	25.57	0.16
19	55/110/275	580 226 (FEAS)	686 397	378.3	644.7	2766.0	19.17	0.65
20	60/120/240	677 683 (FEAS)	816 279	393.2	609.5	2433.1	21.11	0.49
Prosek				173.7	294.0	1723.5	18.72	0.71

3.6 Velike instance

Tabela 3: Rezultati za velike instance ($T = 100$, $k = 3$)

#	$I/J/K$	Ref. (CPLEX)	Best MA	$\bar{t}$ (s)	$\overline{ttot}$ (s)	$\overline{eval} (\times 10^3)$	$\overline{gap}$ (%)	σ (%)
1	70/140/280	741 988 (FEAS)	936 363	296.9	508.4	1499.5	26.96	0.99
2	70/140/350	825 845 (FEAS)	1 036 681	684.7	825.9	2600.4	27.41	1.35
3	80/160/320	851 981 (FEAS)	1 104 103	452.1	844.6	2219.2	31.01	1.77
4	80/160/400	807 109 (FEAS)	1 013 458	738.0	1117.9	2709.3	25.72	0.15
5	90/180/360	801 015 (FEAS)	994 840	289.8	1044.2	2261.4	24.44	0.25
6	90/180/450	953 624 (FEAS)	1 207 206	1035.0	1456.3	3062.8	26.75	0.21
7	100/200/400	1 081 573 (FEAS)	1 415 328	690.1	1832.4	3231.7	31.88	0.90
8	100/200/500	916 397 (FEAS)	1 153 410	1738.0	2921.7	4253.9	27.50	1.16
9	110/220/440	871 261 (FEAS)	1 136 160	938.5	2107.5	3020.3	31.25	0.73
10	110/220/550	945 683 (FEAS)	1 189 702	1466.8	2817.5	4391.2	26.49	0.63
11	120/240/480	971 579 (FEAS)	1 233 209	1238.2	2292.6	3440.7	28.28	1.53
12	120/240/600	1 062 116 (FEAS)	1 412 545	2429.3	4425.9	6408.8	34.85	1.43
13	130/260/520	958 306 (FEAS)	1 207 941	1935.1	4016.1	5183.2	27.06	1.23
14	130/260/650	1 017 071 (FEAS)	1 308 424	2691.6	4560.5	4931.7	29.80	1.01
15	140/280/560	1 015 635 (FEAS)	1 274 963	3843.3	4269.5	4807.6	26.03	0.57
16	140/280/700	1 020 470 (FEAS)	1 260 780	6278.5	7107.7	7383.9	24.58	1.32
17	150/300/600	1 418 601 (FEAS)	1 955 481	5316.6	7644.5	7542.3	39.03	0.86
18	150/300/750	1 125 836 (FEAS)	1 518 030	5697.4	9430.1	8442.3	36.51	1.19
19	160/320/640	1 191 761 (FEAS)	1 519 519	3602.3	6214.8	4998.4	28.05	0.41
20	160/320/800	1 178 469 (FEAS)	1 529 541	7921.3	10665.2	7277.5	30.29	0.67
Prosek				2464.2	3805.2	4483.3	29.19	0.92

3.7 Analiza rezultata

3.7.1 Male instance

Na malim instancama GRASP postiže prosečno odstupanje od $\overline{gap} = 9,24\%$ ($\sigma = 0,48\%$) u odnosu na CPLEX referentnu vrednost, uz $k = 3$ nezavisna izvršavanja po instanci – standardna devijacija je mala (uglavnom $< 1\%$), što ukazuje na konzistentno ponašanje algoritma za fiksirani budžet od $T = 500$ iteracija. Najmanji gap postiže se na instanci 1 (0,67%), najmanjih dimenzija ($4 \times 8 \times 16$), gde algoritam lako pronalazi skoro optimalno rešenje. Najveći gap je na instanci 14 ($12 \times 24 \times 60$, 16,85%), što ukazuje da porast broja

kupaca po DC-u otežava lokalnu pretragu da nadje globalni optimum u zadatom broju iteracija. Prosečno vreme do najboljeg rešenja $\bar{t}$ raste sa dimenzijama instance (od 0,9 s na instanci 1 do 65,5 s na instanci 17), dok je ukupno vreme izvršavanja $\overline{tot}$ svugde ispod 95 sekundi zahvaljujući paralelizaciji.

3.7.2 Srednje instance

Na srednjim instancama prosečno odstupanje iznosi $\overline{gap} = 18,72\%$ ($\sigma = 0,71\%$). Treba napomenuti da su sve CPLEX referentne vrednosti za srednje instance označene statusom FEAS – CPLEX je dostigao vremenski limit od 7200 s bez dokaza optimalnosti, pa referentna vrednost nije nužno optimalna, već najbolje pronađeno dopustivo rešenje kroz snažnu *B&B/cutting-plane* pretragu – suštinski drugačiji (i po vremenu mnogo skuplji) mehanizam od GRASP lokalne pretrage sa $T = 200$ iteracija. Vreme izvršavanja raste sa dimenzijama, od $\overline{tot} = 41,5$ s (instanca 1, $20 \times 40 \times 80$) do $\overline{tot} = 708,0$ s (instanca 17, $50 \times 100 \times 250$), a broj evaluacija funkcije cilja $\overline{eval}$ raste analogno, sa 558 hiljada do preko 3,5 miliona poziva po izvršavanju.

3.7.3 Velike instance

Na velikim instancama prosečno odstupanje iznosi $\overline{gap} = 29,19\%$ ($\sigma = 0,92\%$), nastavljaajući trend rasta uočen kod malih i srednjih grupa ($9,24\% \rightarrow 18,72\% \rightarrow 29,19\%$) – očekivano, budući da je broj iteracija ($T = 100$) najmanji od sve tri grupe, dok je prostor pretrage (I, J, K do 160, 320, 800) najveći. Najmanji gap je na instanci 5 ($90 \times 180 \times 360$, 24,44%), a najveći na instanci 17 ($150 \times 300 \times 600$, 39,03%) – zanimljivo, ne radi se nužno o najvećoj instanci u grupi (instanca 20, $160 \times 320 \times 800$, ima gap 30,29%), što ukazuje da na ovim dimenzijama i dalje postoji varijabilnost zavisna od konkretne strukture instance, ne samo od njene veličine. Vreme izvršavanja raste dramatično sa dimenzijama: $\overline{tot}$ ide od 508,4 s (instanca 1) do preko 10 665 s (instanca 20, skoro 3 sata po izvršavanju), a broj evaluacija funkcije cilja $\overline{eval}$ raste do preko 8,4 miliona poziva po izvršavanju (instanca 18) – uprkos vektorizaciji i paralelizaciji, i dalje se radi o najzahtevnijoj grupi instanci.

3.7.4 Zajednički zaključci

- Odstupanje GRASP-a od CPLEX referentne vrednosti raste sa dimenzijom instance ($9,24\% \rightarrow 18,72\% \rightarrow 29,19\%$), što je očekivana posledica fiksno broja iteracija po grupi ($500 \rightarrow 200 \rightarrow 100$) uz brzo rastući prostor pretrage.
- Standardna devijacija σ je kroz sve tri grupe relativno mala (prosečno 0,48–0,92%, pojedinačno najviše do $\approx 1,8\%$ na par instanci), što pokazuje da je algoritam, uz $k = 3$ izvršavanja po instanci, dovoljno konzistentan – razlike između pojedinačnih izvršavanja iste instance su znatno manje od razlika između instanci.
- Za srednje i velike instance (i deo malih) CPLEX referentne vrednosti same po sebi nisu dokazano optimalne (status FEAS), pa je stvarni gap GRASP-a prema (nepoznatom) optimumu verovatno nešto manji od prikazanog.
- Vektorizacija i paralelizacija (odeljci 2.2.9–2.2.10) su preduslov za to da su $k = 3$ izvršavanja po svih 60 instanci uopšte bila izvodljiva u razumnom vremenu – pre tih izmena, jedno izvršavanje na srednjoj/velikoj instanci je trajalo od nekoliko minuta do više sati, a za velike instance ($T = 100$, $k = 3$, 20 instanci) ukupno izvršavanje je i pored ubrzanja trajalo više desetina sati.

4 Zaključak

U ovom radu prezentovana je implementacija GRASP metaheuristike za Dvostepeni transportni problem sa fiksnim troškovima (TS-FCTP). Algoritam kombinuje pohlepno-nasumičnu konstrukciju sa reaktivnim parametrom α , VND lokalnu pretragu sa četiri susedstva, elitni bazen, path relinking (unija i presek skupova otvorenih DC-ova) i shake perturbaciju.

Centralna arhitekturna odluka – direktna pretraga u prostoru y -tokova uz rekonstrukciju x -tokova procedurom REPAIR – efikasno smanjuje dimenzionalnost problema. Nad tom osnovom, implementacija je dodatno **vektorizovana** (NumPy, umesto Python petlji po I, J, K) i **paralelizovana** (multiprocessing, batch nezavisnih GRASP iteracija) – što je omogućilo da se svaka instanca izvrši $k = 3$ puta i dobiju statistički smislene ocene ($\overline{gap}$, σ), umesto ranijeg pristupa sa $k = 1$ zbog neizvodljivo dugih vremena izvršavanja. Usput su ispravljene i dve greške u infrastrukturi za MA izveštaj: t_i (vreme do prvog nalaska najboljeg rešenja) se ranije nije stvarno pratilo (bilo je jednako ukupnom vremenu), a referentna CPLEX vrednost se zbog pogrešne putanje/poređenja imena fajla skoro nikad nije učitala, pa se gap u praksi računao prema sopstvenom najboljem rezultatu umesto prema CPLEX optimumu.

Testiranje na svih 60 instanci ($k = 3$ izvršavanja po instanci) pokazuje da GRASP postiže dobra rešenja na malim instancama (prosečan gap 9,24%, $\sigma = 0,48\%$), sa umerenim padom kvaliteta na srednjim instancama (gap 18,72%, $\sigma = 0,71\%$) i daljim padom na velikim instancama (gap 29,19%, $\sigma = 0,92\%$), usled fiksnog i sve manjeg broja iteracija ($500 \rightarrow 200 \rightarrow 100$) u odnosu na sve veći prostor pretrage na krupnijim problemima. Kroz sve tri grupe standardna devijacija ostaje relativno mala, što ukazuje na konzistentno ponašanje algoritma nezavisno od dimenzije instance.

Mogući pravci daljeg unapređenja:

- Povećati broj iteracija T na srednjim i velikim instancama (npr. dodatnim ubrzanjem REPAIR-a ili doniranjem više vremena izvršavanju) kako bi se smanjio uočeni gap na tim grupama;
- Uvesti mehanizam kešfiguriranja (*caching*) funkcije cilja za ponovljena stanja, čime bi se omogućilo praćenje $cache_i/cache\%$ iz specifikacije izveštaja i potencijalno smanjio broj stvarnih evaluacija;
- Inkrementalna (a ne potpuna) rekonstrukcija x -tokova u REPAIR-u za pogođene DC-ove, uz pažljivu proveru da ne naruši dopustivost usled tačno izbalansiranog kapaciteta;
- Ugradnja intenzivnijeg path relinking-a koji istražuje i međutačke na putu između dva elitna rešenja, ne samo krajnje uniju i presek;

Literatura

- [1] H. I. Calvete, C. Galé, J. A. Iranzo, P. Toth, *A matheuristic for the two-stage fixed-charge transportation problem*.